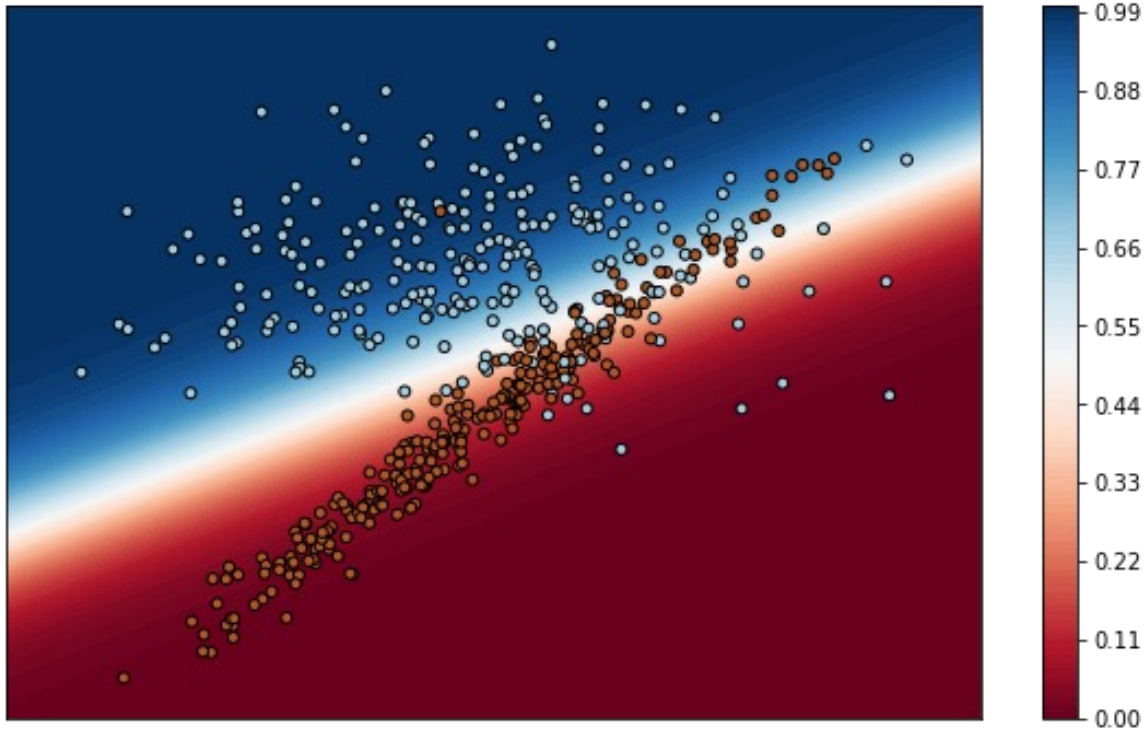
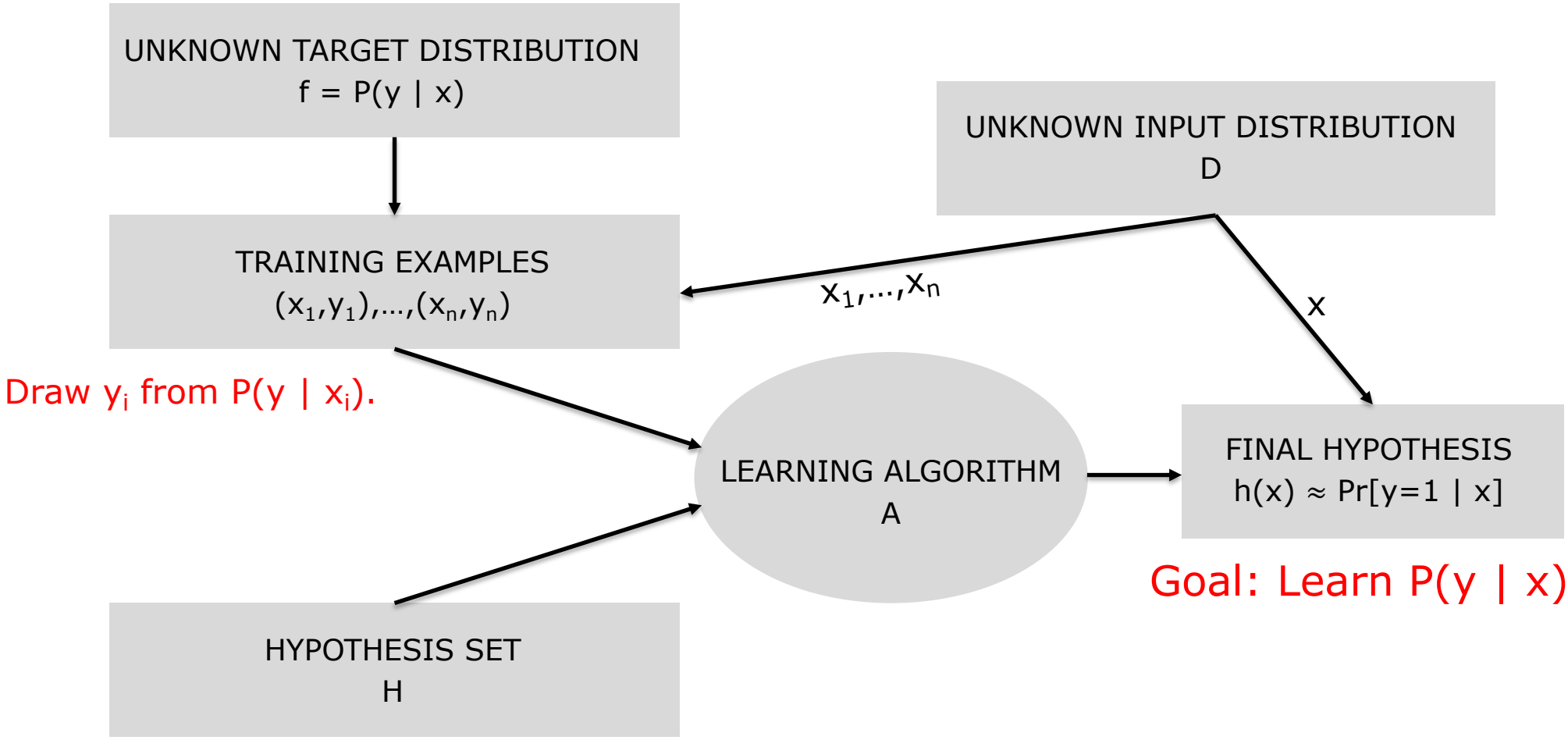
Softmax for Multinomial Logistic Regression

Logistic Regression

- Logistic Regression:
  - Predict(x)= $\theta(w^T x)= \frac{1}{1+e^{-w^T x}}$
  - Want $\theta(w^T x)=Pr[y=1|x]$.
  - Learn P(y|x).



Logistic Regression



Logistic Regression

- Likelihood of training data.
  - Hypothesis w, i.e. $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
- Likelihood of data labels with $S=(x_1,y_1),\ldots,(x_n,y_n)$ for w:
  - $\Pr[S \mid w,X] = \prod_{i=1}^{n} \frac{1}{1+e^{-y_i w^T x_i}}$

- Maximum Likelihood:
  - Maximize Pr[S | w,X] same as minimize –ln(Pr[S | w,X]).
  - Showed –ln(Pr[S | w,X]) is convex.
  - Computed gradient of –ln(Pr[S | w,X]).
  - Use Gradient Descent to minimize.

Multinomial Regression

- Logistic Regression
  - Solution last time only for two classes {-1,1}.
  - What about K classes?







  - Red, white or rose?

Multinomial Regression

- Logistic Regression
  - Solution last time only for two classes {-1,1}.
  - What about K classes?



  - Want to learn P(y | x), where P(y | x) is a distribution over K classes.

Multinomial Regression

- Representing Labels
  - First question is how do we represent labels from a set of K classes?
  - Natural idea:
    - Assign numbers 1,…,K.
  - Downside:
    - Induces an ordering on labels.
    - 1 and 2 are ”closer” than 1 and K.



One-in-K Encoding

- Representing Labels
  - First question is how do we represent labels from a set of K classes?
  - Represent labels as vectors:
    - K-dimensional vector.
    - One coordinate per class.
    - If label i, put a 1 in i'th coordinate and 0 in others.
  - Red: (1, 0, 0)
  - White: (0, 1, 0)
  - Rose: (0, 0, 1)



One-in-K Encoding

- Two classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$
- Each y_i in {-1,1}

- K classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}$
- Each row of Y has a single ”1” and K-1 ”0”s.



One-in-K Encoding

- Logistic Regression two classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$
- $\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$ should equal Pr[y=1|x].
  - Also learn Pr[y=-1|x] = 1-Pr[y=1|x].

- K classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}$
- Want to learn Pr[y=i|x] for i=1,...,K.
  - Learn P(y|x).







Multinomial Regression

- K classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}$
- Want to learn Pr[y=i|x] for i=1,...,K.
  - Learn P(y|x).

- Linear Models:
  - Want to output something for each class.
  - Should give the probability.
  - Use a weight matrix W rather than vector w.







Multinomial Regression

- K classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Want to learn Pr[y=i|x] for i=1,...,K.
  - Learn P(y|x).

- Linear Models:
  - Want to output something for each class.
  - Should give the probability.
  - Use a weight matrix W rather than vector w.







Multinomial Regression

- K classes, single data point x:
  - $x = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Before (two classes):
  - Estimates based on $x^T w$ with $w = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix}$
- Now (K classes):
  - Base estimates on $x^T W$:
    - $x^T W = \begin{pmatrix} x^T w_{:,1} & \cdots & x^T w_{:,K} \end{pmatrix}$

$w_{:,j}$ is j'th column of weight matrix W.





Multinomial Regression

- K classes, single data point x:
  - $x = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Before (two classes):
  - Estimates based on $x^T w$ with $w = \begin{pmatrix} w_1 \\ \vdots \\ w_d \end{pmatrix}$
- Now (K classes):
  - Base estimates on $x^T W$:
    - $x^T W = \begin{pmatrix} x^T w_{:,1} & \cdots & x^T w_{:,K} \end{pmatrix}$

Like having K linear models, one for each class.



Multinomial Regression

- K classes, single data point x:
  - $x = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Base estimates on $x^T W$:
  - $x^T W = \begin{pmatrix} x^T w_{:,1} & \cdots & x^T w_{:,K} \end{pmatrix}$

- Want to map $x^T W$ to a probability distribution.
- Inspired by sigmoid:
  - For vector $z = (z_1, \ldots, z_K)$
    - softmax(z) is a vector of probabilities.
    - softmax(z)$_j$= $\frac{e^{z_j}}{\sum_{i=1}^{K} e^{z_i}}$

Normalization so sum to 1.



Multinomial Regression

- For vector $z = (z_1, \ldots, z_K)$
  - softmax(z)$_j$= $\frac{e^{z_j}}{\sum_{i=1}^{K} e^{z_i}}$

- Example:
  - z=(3, 2, 4)
  - softmax(z)=(e^3/(e^2+e^3+e^4),e^2/(e^2+e^3+e^4),e^4/(e^2+e^3+e^4))
  - softmax(z)=(0.245, 0.090, 0.665)

Normalization so sum to 1.



Multinomial Regression

- K classes, single data point x:
  - $x = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Base estimates on $x^T W$:
  - $x^T W = \begin{pmatrix} x^T w_{:,1} & \cdots & x^T w_{:,K} \end{pmatrix}$
- Final estimates/outputs:
  - softmax($x^T W$).



Multinomial Regression

- K classes, n data points $x_1,\ldots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Final estimates/outputs
  - $XW = \begin{pmatrix} x_1^T w_{:,1} & \cdots & x_1^T w_{:,K} \\ \vdots & \ddots & \vdots \\ x_n^T w_{:,1} & \cdots & x_n^T w_{:,K} \end{pmatrix}$
  - Predict(X)=softmax(XW) //apply softmax row-wise.



Multinomial Regression

Recall one 1 pr. row, rest 0.

Indicates the class.

- K classes, n data points $x_1,\dots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Final estimates/outputs
  - $XW = \begin{pmatrix} x_1^T w_{:,1} & \cdots & x_1^T w_{:,K} \\ \vdots & \ddots & \vdots \\ x_n^T w_{:,1} & \cdots & x_n^T w_{:,K} \end{pmatrix}$
  - Predict(X)=softmax(XW) //apply softmax row-wise.
- Like logistic regression:
  - Do maximum likelihood.



Multinomial Regression

Recall one 1 pr. row, rest 0.

Indicates the class.

- K classes, n data points $x_1,\dots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Final estimates/outputs
  - $XW = \begin{pmatrix} x_1^T w_{:,1} & \cdots & x_1^T w_{:,K} \\ \vdots & \ddots & \vdots \\ x_n^T w_{:,1} & \cdots & x_n^T w_{:,K} \end{pmatrix}$
  - Predict(X)=softmax(XW) //apply softmax row-wise.
- Assume Predict(X) gives true probabilities.
  - Likelihood of data (labels)?
    - S=$(x_1,y_1),\dots,(x_n,y_n)$



Multinomial Regression

Recall one 1 pr. row, rest 0.

- K classes, n data points $x_1,\ldots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Assume Predict(X)=softmax(XW) gives true probabilities.
  - Likelihood of data (labels)?
    - $S=(x_1,y_1),\ldots,(x_n,y_n)$

- $\Pr[S|X,W]=\prod_{i=1}^{n}\Pr[y_i|x_i,W]=$
- $\prod_{i=1}^{n} y_i^T \mathrm{softmax}(x_i^T W)$

- Maximize over W!

Inner product with y_i extracts just entry j with $y_{i,j}=1$.






Multinomial Regression

- K classes, n data points $x_1,\dots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Maximize over W
  - $\prod_{i=1}^{n} y_i^T \text{softmax}(x_i^T W)$
- Negative log likelihood. Scale by 1/n. Minimize:
  - $E_{in}(W) = -\frac{1}{n}\sum_{i=1}^{n} \ln(y_i^T \text{softmax}(x_i^T W))$
- Can be shown to be convex!
- We can do gradient descent if we can compute the gradient $\nabla E_{in}(W)$.



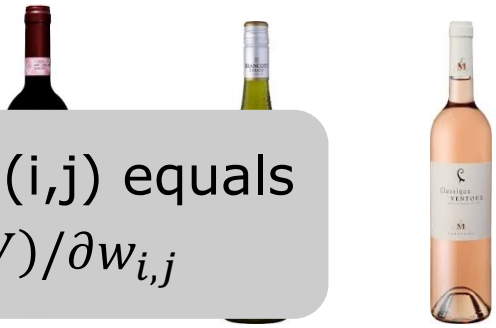
Multinomial Regression

- K classes, n data points $x_1,\ldots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Maximize over W
  - $\prod_{i=1}^{n} y_i^T \text{softmax}(x_i^T W)$
- Negative log likelihood. Scale by 1/n. Min
  - $E_{in}(W) = -\frac{1}{n}\sum_{i=1}^{n} \ln(y_i^T \text{softmax}(x_i^T W))$
- Luckily, we are told that the gradient is
  - $\nabla E_{in}(W) = -\frac{1}{n} X^T (Y - \text{softmax}(XW))$

Sanity check:

Gradient should have same shape as W

Entry (i,j) equals $\partial E_{in}(W)/\partial w_{i,j}$



Multinomial Regression

- K classes, n data points $x_1,\ldots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Maximize over W
  - $\prod_{i=1}^{n} y_i^T \text{softmax}(x_i^T W)$
- Negative log likelihood. Scale by 1/n. Min
  - $E_{in}(W) = -\frac{1}{n}\sum_{i=1}^{n} \ln(y_i^T \text{softmax}(x_i^T W))$
- Luckily, we are told that the gradient is
  - $\nabla E_{in}(W) = -\frac{1}{n} X^T (Y - \text{softmax}(XW))$

dxK dxn nxK nxK

Sanity check:

Gradient should have same shape as W



Gradient Descent

- Gradient Descent for Multinomial Regression:
  - Initialize W (e.g. random N(0,1) distributed entries).
  - While more time:
    - $\nabla \mathrm{E}_{\mathrm{in}}(W) = -\frac{1}{\mathrm{n}} \mathrm{X}^{\mathrm{T}}(\mathrm{Y} - \mathrm{softmax}(XW))$
    - W = W - $\eta \nabla E_{in}(W)$
  - Return W.

Stochastic Gradient Descent

- Mini-batch SGD for Multinomial Regression:
  - Initialize W (e.g. random N(0,1) distributed entries).
  - While more time:
    - Permute $(x_1,y_1),\ldots,(x_n,y_n)$ randomly.
    - For i=1,...,n/B:
      - Let $(x_1',y_1'),\ldots,(x_B',y_B')$ be next B data items.
      - $G = -\frac{1}{B} X'^T(Y' - \text{softmax}(X'W))$
      - $W = W - \eta G$
  - Return W.

- Each time we permute is called an epoch.
- Practical: Reduce η by a small factor after fixed number of epochs.

Softmax in Practice

- For vector $z = (z_1, \dots, z_K)$
  - softmax(z)$_j$= $\frac{e^{z_j}}{\sum_{i=1}^{K} e^{z_i}}$

- Issue:
  - Not very numerically stable.
  - Exponentials become large numbers.
  - Lose precision when summing them.
    - Floating point representations on computers.

Softmax in Practice

- For vector $z = (z_1, \dots, z_K)$
  - softmax(z)$_j$= $\frac{e^{z_j}}{\sum_{i=1}^{K} e^{z_i}}$

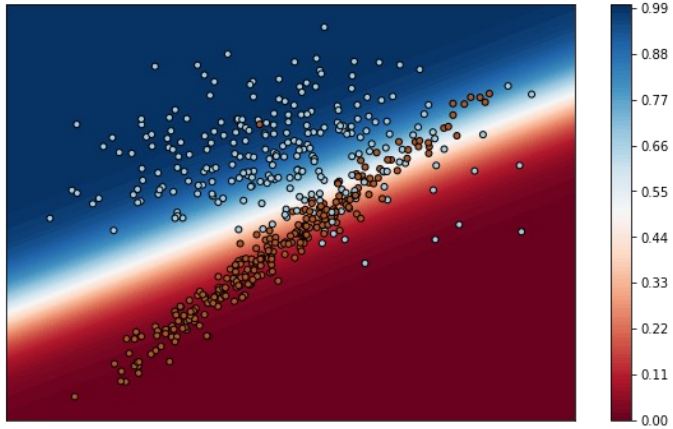
- Practical Fix/Help:
  - Rewrite to avoid large numbers:
  - $\frac{e^{z_j}}{\sum_{i=1}^{K} e^{z_i}} = e^{z_j - \ln(\sum_{i=1}^{K} e^{z_i})} =$
  - Still sum large numbers inside ln() unfortunately.
  - Let z_a be largest among $z_1,\dots,z_K$. Then equals:
  - $e^{z_j - \ln(e^{z_a} \sum_{i=1}^{K} e^{z_i - z_a})} = e^{z_j - z_a - \ln(\sum_{i=1}^{K} e^{z_i - z_a})}$

  - Now all summands less than 1. Helps a lot for precision.

Nonlinear Transforms

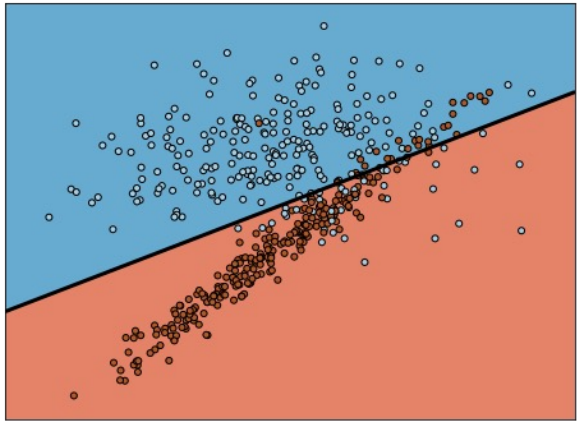
Linear Models

$\theta(w^T x) = \frac{1}{1+e^{-w^T x}}$



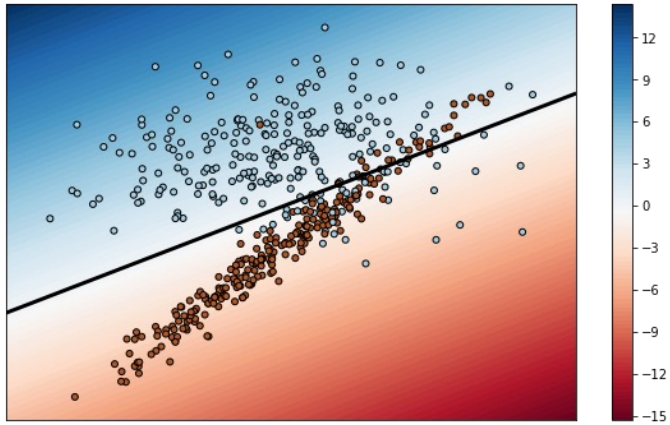
Logistic Regression

$\text{sign}(w^T x)$



Classification

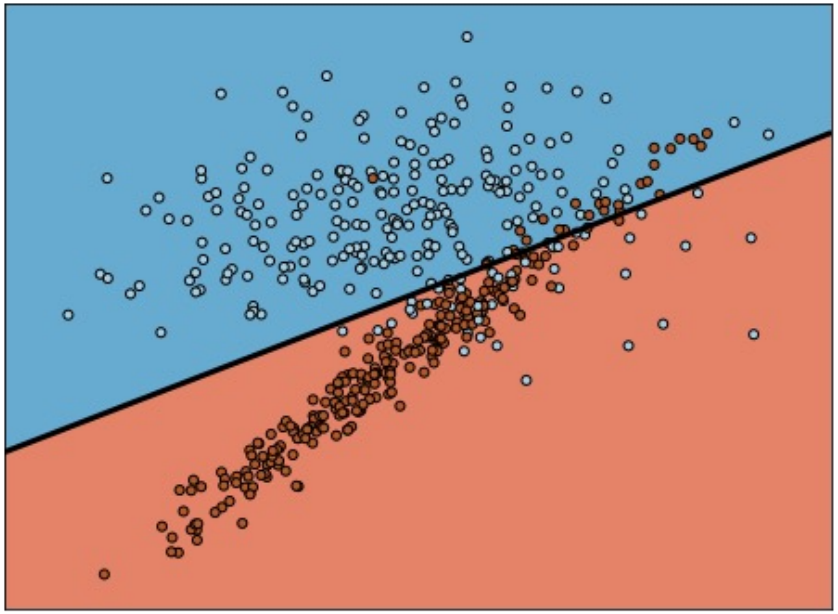
$w^T x$



Regression

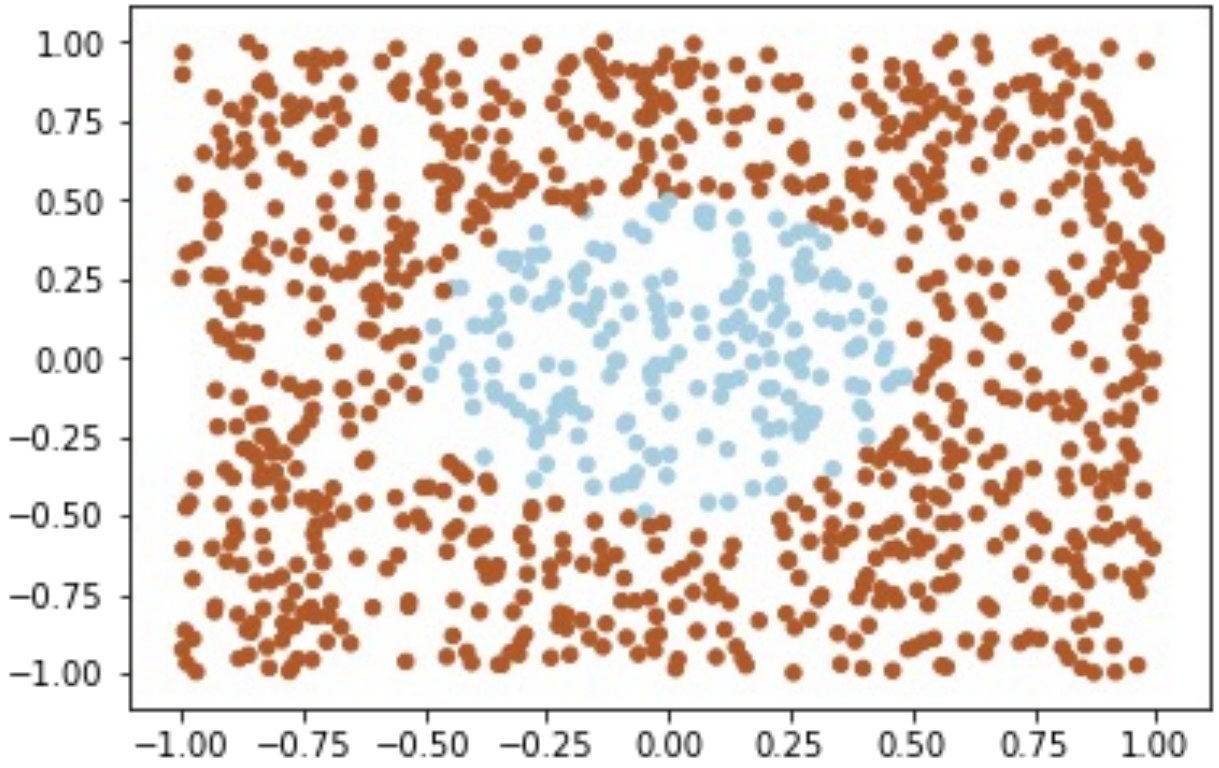
Linear Models

- Common:
  - All based on $w^T x$.
  - Need a hyperplane to separate/describe the data well.



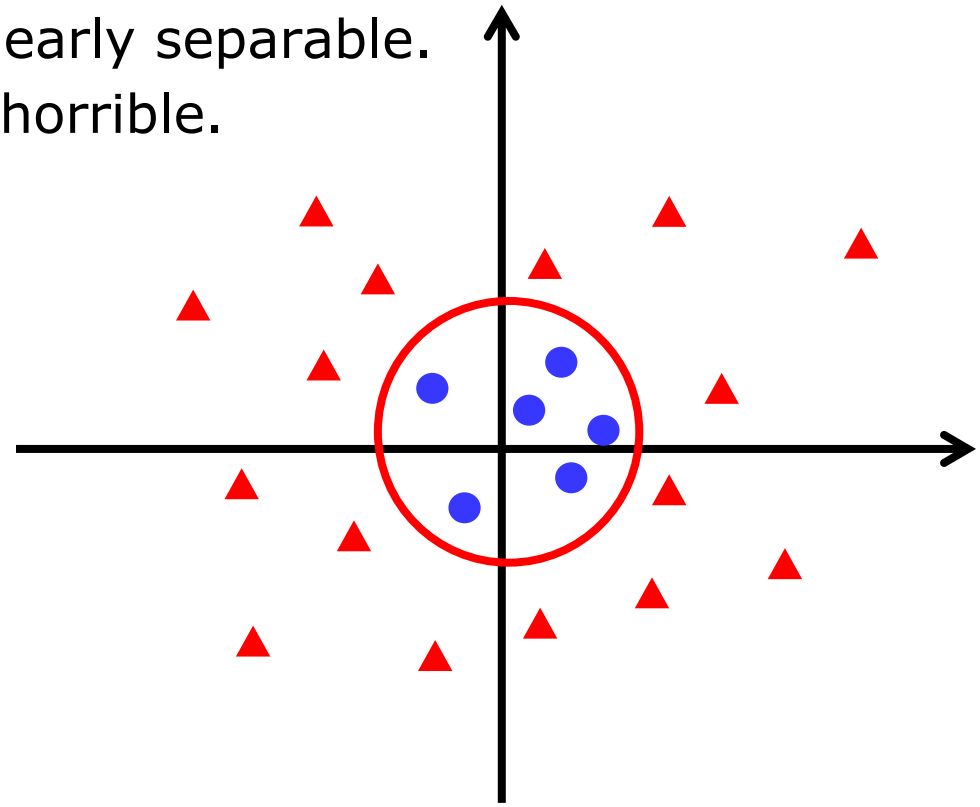
Linear Models

- Nonlinear Data:
  - Just abandon linear models?



Linear Models

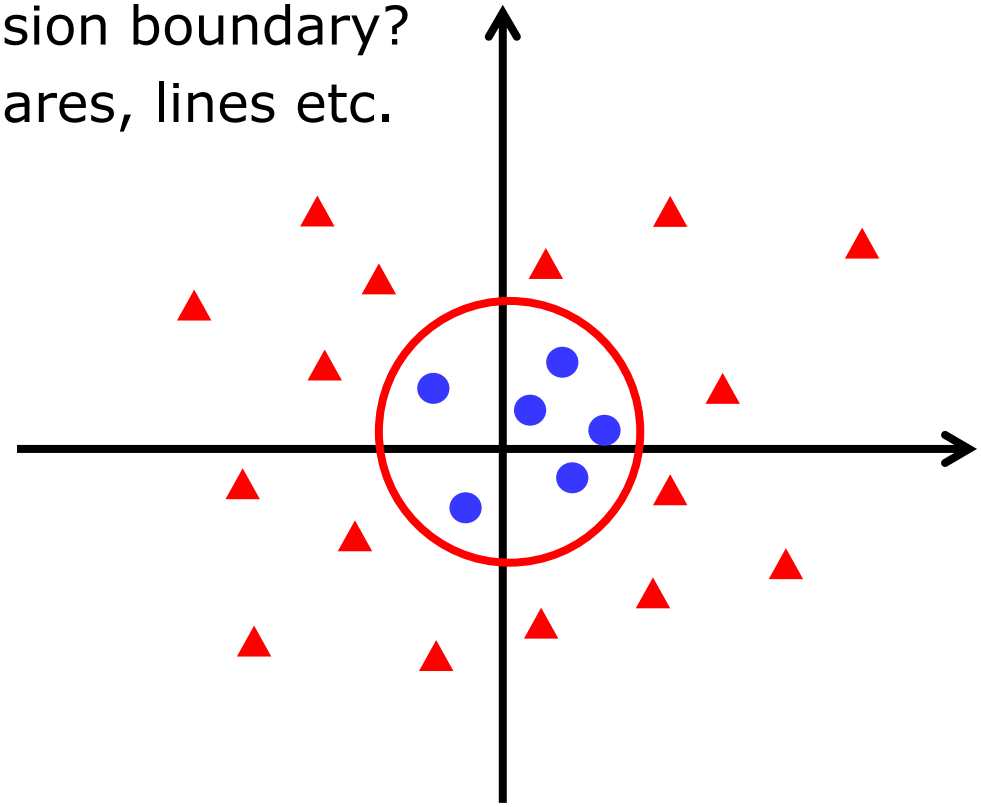
- Example:
  - Far from linearly separable.
  - Perceptron horrible.



- Looks like distance from origin could classify well.

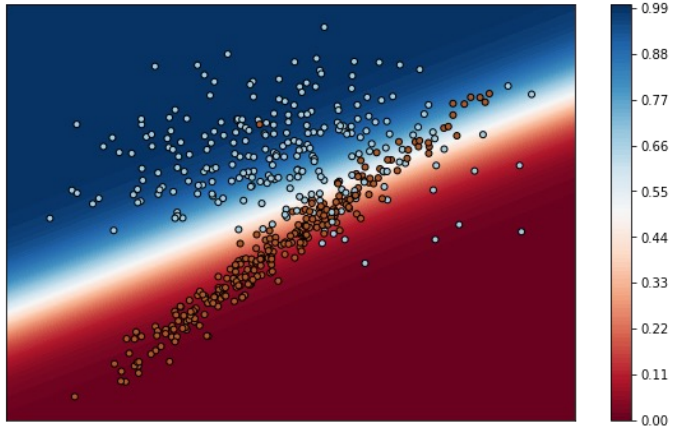
Linear Models

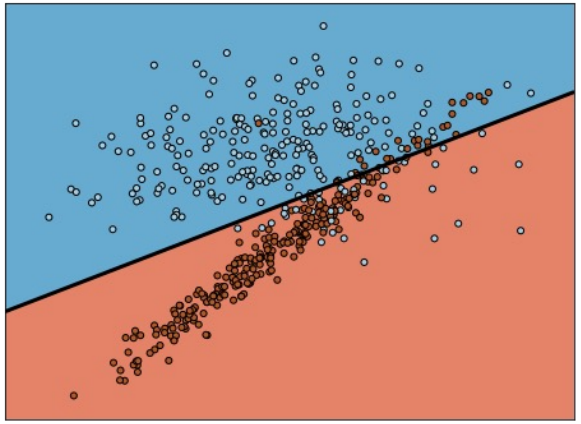
- Do we need new algorithms and techniques for every reasonable shape of a decision boundary?
  - Circles, squares, lines etc.

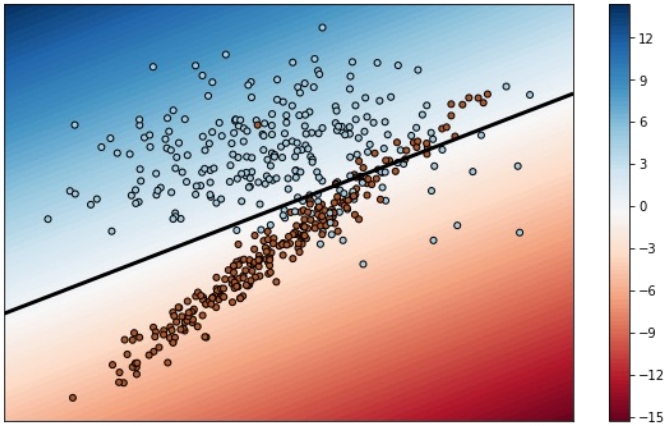


Linear Models

- Close inspection.
  - All the algorithms for linear models treat X and y as constants. Only w changes during the algorithm.
  - We could change X and still run algorithms!





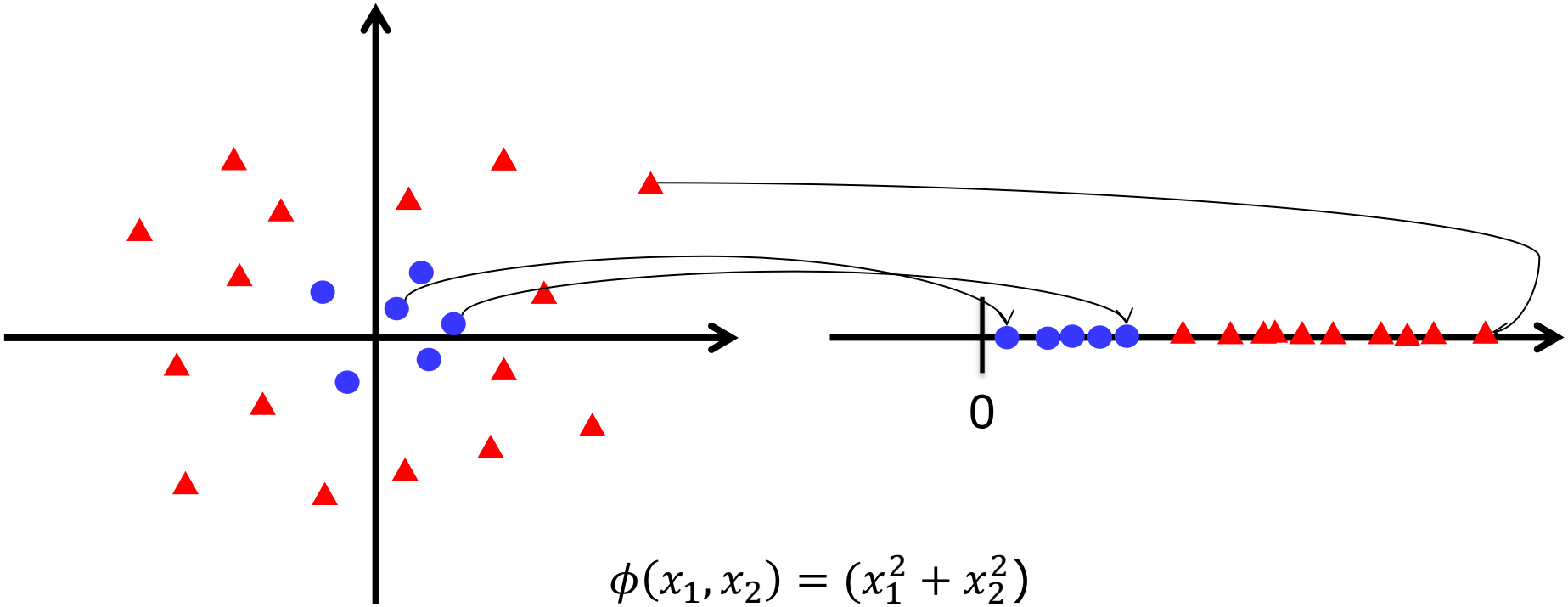


Feature Tran

Feature transform

Feature space. Often denoted Z

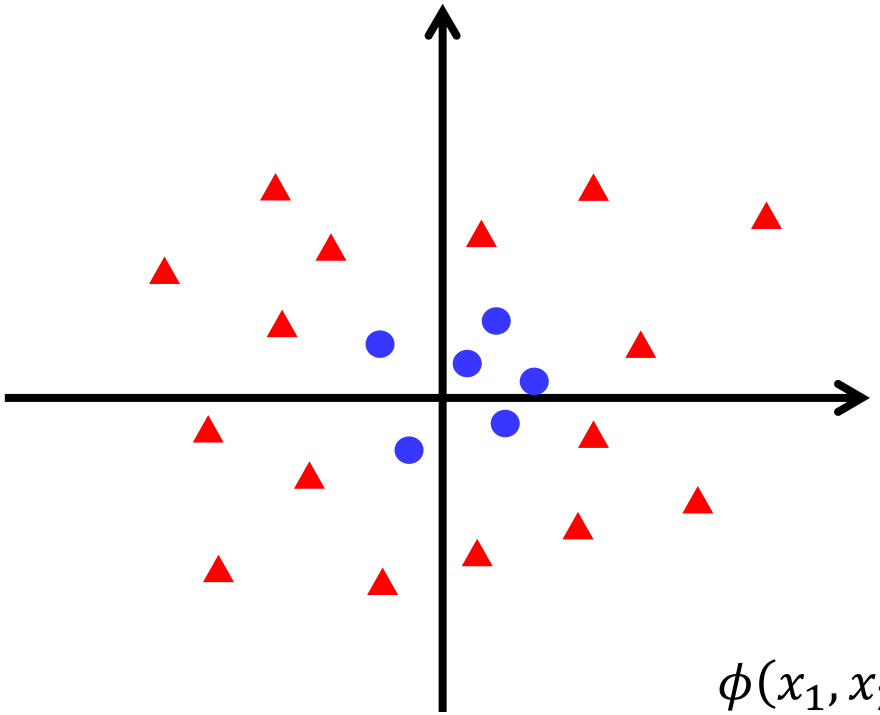
- Nonlinear transforms:
  - Let $\phi : R^d \rightarrow R^{d'}$ be any mapping from R^d to $R^{d'}$.
  - Apply ϕ to all feature vectors before training linear models.
  - Get new data matrix X' which is n x d' instead of n x d.



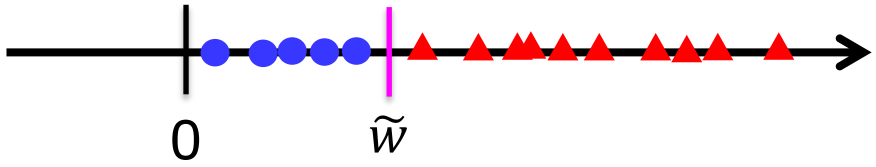
$$\phi(x_1, x_2) = (x_1^2 + x_2^2)$$

Feature Transforms

- Nonlinear transforms:
  - Let $\phi : R^d \rightarrow R^{d'}$ be any mapping from R^d to $R^{d'}$.
  - Apply ϕ to all feature vectors before training linear models.
  - Get new data matrix X' which is n x d' instead of n x d.



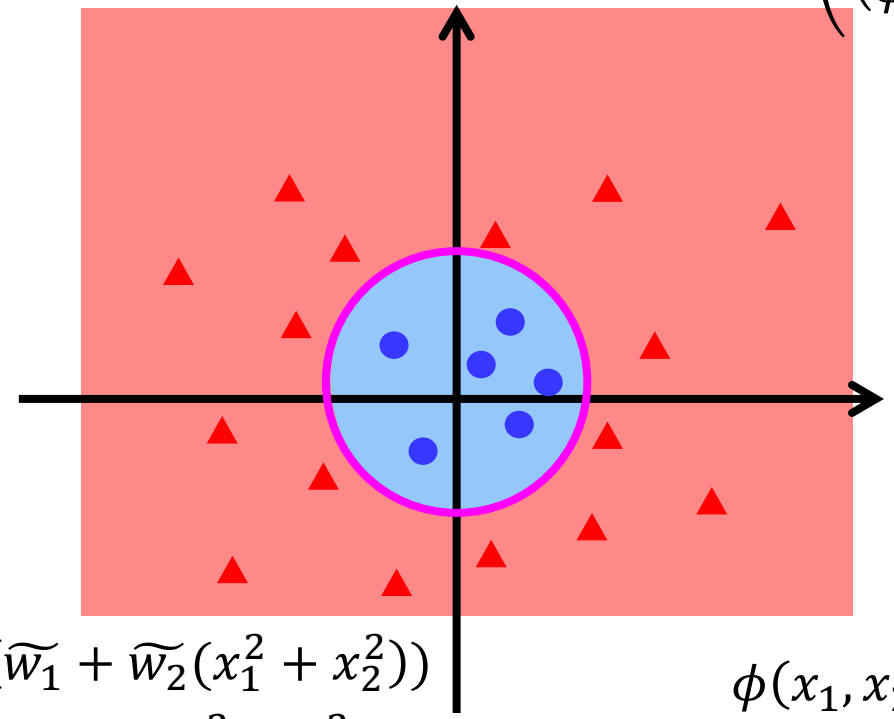
Run Perceptron on X' and find separating hyperplane $\tilde{w}$.



$$\phi(x_1, x_2) = (x_1^2 + x_2^2)$$

Feature Transforms

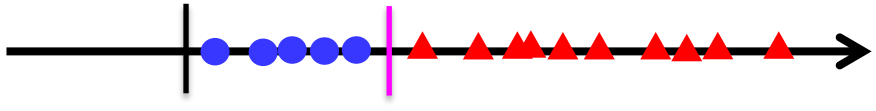
- Perceptron:
  - Recall prepends an all-1s column to X' to allow a bias.
  - Effective hypothesis: $\text{sign}\left(\begin{pmatrix}1\\ \phi(x_1,x_2)\end{pmatrix}^T \widetilde{w}\right)$



$\text{sign}(\widetilde{w_1} + \widetilde{w_2}(x_1^2 + x_2^2))$

$\text{sign}(7 - 1(x_1^2 + x_2^2))$

Run Perceptron on X' and find separating hyperplane $\widetilde{w}$.



$\widetilde{w} = (\widetilde{w}_1, \widetilde{w}_2)$

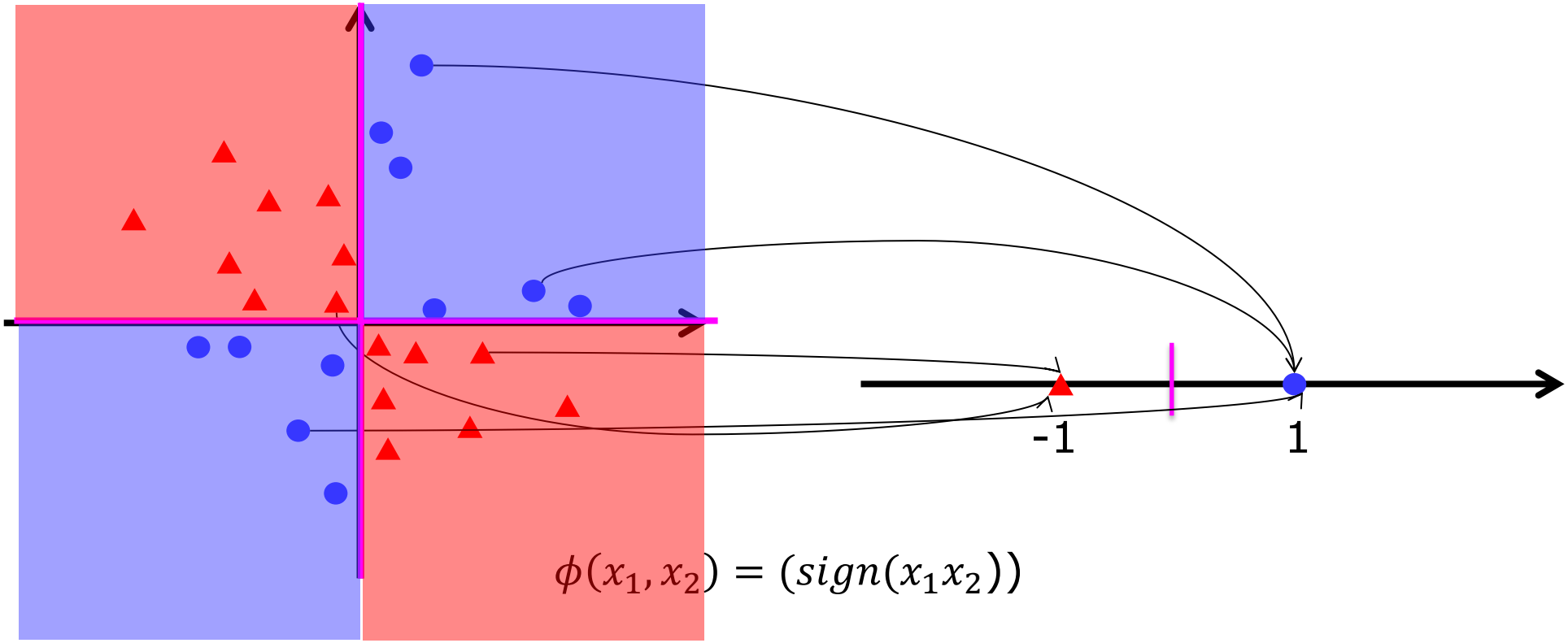
$\widetilde{w} = (7, -1)$

$\phi(x_1, x_2) = (x_1^2 + x_2^2)$

Bias

Feature Transforms

- Another Example:

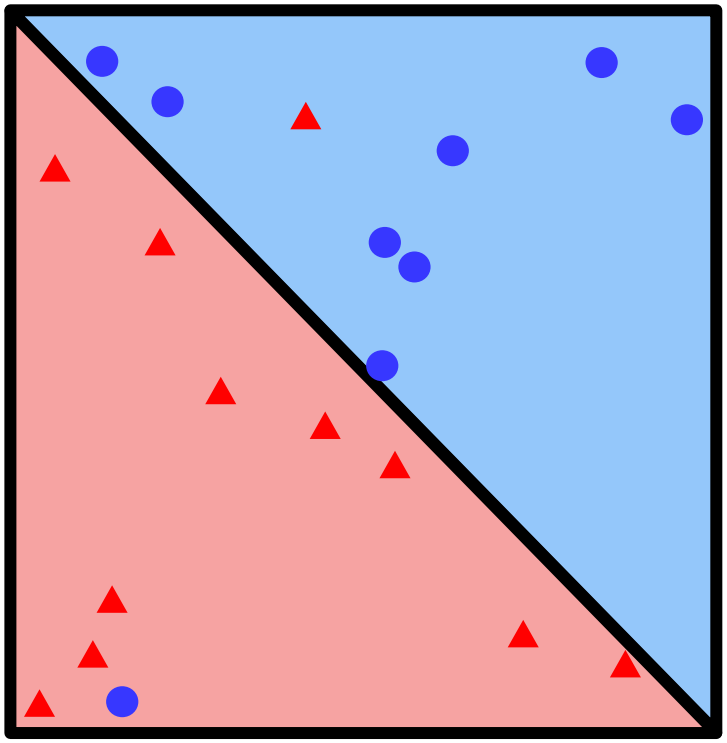


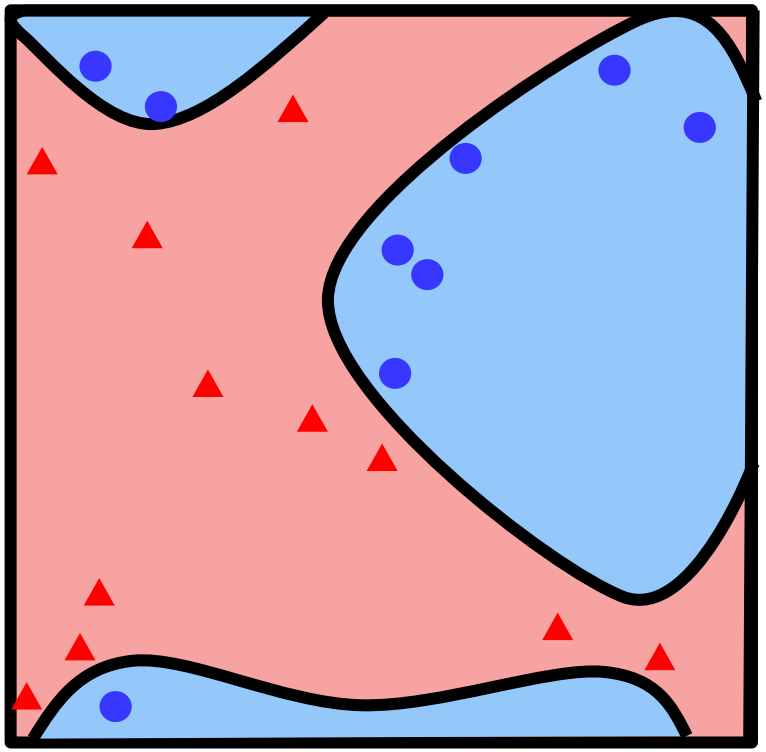
Some Considerations

- Pitfalls:
  - We saw in learning theory that the complexity of the hypothesis set H matters for generalization.
    - Low complexity better. |H|=M.
      - $\Pr[|E_{in}(h^*) - E_{out}(h^*)| > \varepsilon] \leq 2Me^{-2\varepsilon^2 n}$
    - Argued for d floats representing w, $M=2^{32d}$
  - So, should avoid too many features after transform.
    - E.g. might be bad to have:
    - $\phi(x_1,x_2)=(1,x_1,x_2,x_1^2,x_2^2,x_1^3,x_2^3,x_1x_2,x_1x_2^2,\ldots)$.

Some Considerations

4th degree transform
$(1, x_1, x_2, \ldots, x_1^3 x_2, x_1^4, \ldots)$





Probably large E_{out}

- Not worth high complexity to decrease E_{in} a tiny bit.

Some Considerations

- Pitfalls:
  - We saw in learning theory that the complexity of the hypothesis set H matters for generalization.
    - Low complexity better. |H|=M.
      - $\Pr[|E_{in}(h^*) - E_{out}(h^*)| > \varepsilon] \leq 2Me^{-2\varepsilon^2 n}$
    - Argued for d floats representing w, $M=2^{32d}$
  - So, should avoid too many features after transform.
    - E.g. might be bad to have:
    - $\phi(x_1,x_2)=(1,x_1,x_2,x_1^2,x_2^2,x_1^3,x_2^3,x_1x_2,x_1x_2^2,\ldots)$.
  - In previous examples, stared at data and then chose transform:
    - Bad, since Hoeffding only holds if choose H before seeing the data.

Some Considerations

- Pitfalls:
  - We saw in learning theory that the complexity of the hypothesis set H matters for generalization.
    - Low complexity better. |H|=M.
      - $\Pr[|E_{in}(h^*) - E_{out}(h^*)| > \varepsilon] \leq 2Me^{-2\varepsilon^2 n}$
    - Argued for d floats representing w, $M=2^{32d}$
  - So, should avoid too many features after transform.
    - E.g. might be bad to have:
    - $\phi(x_1,x_2)=(1,x_1,x_2,x_1^2,x_2^2,x_1^3,x_2^3,x_1x_2,x_1x_2^2,\ldots)$.
  - In previous examples, stared at data and then chose transform:
    - Intuitively, when you look at data, your H is actually all the hypotheses you could have ever thought of.

Some Considerations

- Pitfalls:
  - We saw in learning theory that the complexity of the hypothesis set H matters for generalization.
    - Low complexity better. |H|=M.
      - $\Pr[|E_{in}(h^*) - E_{out}(h^*)| > \varepsilon] \leq 2Me^{-2\varepsilon^2 n}$
    - Argued for d floats representing w, $M=2^{32d}$
  - So, should avoid too many features after transform.
    - E.g. might be bad to have:
    - $\phi(x_1,x_2)=(1,x_1,x_2,x_1^2,x_2^2,x_1^3,x_2^3,x_1x_2,x_1x_2^2,\ldots)$.
  - If have domain knowledge and you think you have a useful feature, you can include it *before* looking at the data.
    - Experts may handcraft features.
      - Just don't look at the data first (if you want theoretical guarantees).